

- Apostila – Orientação a Objetos em Java
 - 1. Introdução à Orientação a Objetos
 - Por que usar OO?
 - 2. Classe e Objeto
 - 3. Estrutura de uma Classe em Java
 - 4. Encapsulamento
 - 5. Abstração
 - 6. Herança
 - 7. Polimorfismo
 - 8. Interfaces
 - 9. Composição vs Herança
 - 10. Acoplamento e Coesão
 - Acoplamento
 - Coesão
 - Relação entre Acoplamento e Coesão
 - 11. Erros Comuns em OO
 - 12. OO e SOLID
 - Exemplos práticos (Feira Livre)
 - SRP — Single Responsibility Principle
 - OCP — Open-Closed Principle
 - LSP — Liskov Substitution Principle
 - ISP — Interface Segregation Principle
 - DIP — Dependency Inversion Principle
 - 13. OO e Arquitetura de Software
 - 14. UML Essencial – Notação e Dicas
 - Diagramas mais úteis na disciplina
 - Notações essenciais (Diagrama de Classes)
 - Mapeamento UML → Código (Java)
 - Exemplo simples (Feira)
 - Diagrama de Sequência – exemplo (Fluxo de Finalização de Pedido)
 - Quando usar UML
 - 15. Estudo de Caso Conceitual – Feira Livre
 - 16. Quiz de Revisão – Orientação a Objetos, Acoplamento e Coesão
 - Instruções
 - Questão 1
 - Questão 2
 - Questão 3

- [Questão 4](#)
- [Questão 5](#)
- [Questão 6](#)
- [Questão 7](#)
- [Questão 8](#)
- [Questão 9](#)
- [Questão 10](#)
- [17. Gabarito do Quiz](#)
- [18. Conclusão](#)
- [19. Bibliografia Recomendada](#)

Apostila – Orientação a Objetos em Java

Orientação a Objetos em Java

Encapsulamento · Abstração · Herança · Polimorfismo · Interfaces

Disciplina: Projeto e Arquitetura de Sistemas

Objetivo: Fornecer base conceitual sólida de Orientação a Objetos (OO – Orientação a Objetos) para apoiar o estudo de SOLID (Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion), GRASP (General Responsibility Assignment Software Patterns – Padrões de Atribuição de Responsabilidade), Padrões de Projeto e Arquitetura de Software.

1. Introdução à Orientação a Objetos

A Orientação a Objetos (OO) é um paradigma de programação baseado na ideia de representar sistemas por meio de **objetos**, que são entidades que possuem **estado** (dados) e **comportamento** (operações).

Diferente da programação procedural, onde o foco está em funções e processos, na OO o foco está em **modelar o domínio do problema** por meio de classes e objetos.

Por que usar OO?

- Facilita a modelagem de problemas complexos
 - Estimula reutilização de código
 - Favorece manutenção e evolução
 - Base para arquiteturas modernas
-

2. Classe e Objeto

- **Classe:** é um molde (blueprint) que define as características de um tipo de objeto.
- **Objeto:** é uma instância concreta de uma classe.

Exemplo conceitual (domínio da feira):

- Classe: **Feirante**
- Objetos: João, Maria, Ana

Em Java:

```
public class Feirante {  
    private String nome;  
}
```

3. Estrutura de uma Classe em Java

Uma classe em Java normalmente contém:

- Atributos (campos)
- Construtores
- Métodos

Exemplo:

```
public class Produto {  
    private String nome;  
    private double preco;  
  
    public Produto(String nome, double preco) {  
        this.nome = nome;  
        this.preco = preco;  
    }  
  
    public double getPreco() {  
        return preco;  
    }  
}
```

4. Encapsulamento

Encapsulamento é o princípio de **ocultar os detalhes internos** de uma classe e expor apenas o que é necessário.

Em Java, isso é feito por meio dos modificadores de acesso:

- `private`
- `protected`
- `public`

Benefícios:

- Reduz acoplamento
- Aumenta segurança
- Facilita manutenção

Exemplo em Java (encapsulamento com validação):

```
public class Produto {  
    private String nome;  
    private double preco;  
  
    public Produto(String nome, double preco) {  
        this.nome = nome;  
        setPreco(preco);  
    }  
}
```

```
public String getNome() {  
    return nome;  
}  
  
public double getPreco() {  
    return preco;  
}  
  
public void setPreco(double preco) {  
    if (preco < 0) {  
        throw new IllegalArgumentException("Preço não pode ser negativo");  
    }  
    this.preco = preco;  
}  
  
public double aplicarDesconto(double percentual) {  
    // Regra encapsulada: desconto máximo de 50%  
    if (percentual < 0 || percentual > 0.5) {  
        throw new IllegalArgumentException("Percentual inválido");  
    }  
    return this.preco * (1 - percentual);  
}  
}
```

5. Abstração

Abstração consiste em **focar no essencial** e ignorar detalhes irrelevantes para o contexto.

Exemplo:

- Um **Produto** tem nome e preço
- Não importa como o preço é calculado internamente

A abstração permite trabalhar com **modelos conceituais**, não com detalhes de implementação.

6. Herança

Herança é um mecanismo que permite que uma classe herde características de outra.

Exemplo:

```
public class ProdutoOrganico extends Produto {  
}
```

A herança representa a relação "é um".

⚠ Cuidado: herança em excesso gera sistemas rígidos e difíceis de manter.

7. Polimorfismo

Polimorfismo permite que objetos diferentes respondam à mesma mensagem de formas distintas.

```
public class ProdutoOrganico extends Produto {  
    public ProdutoOrganico(String nome, double preco) {  
        super(nome, preco);  
    }  
  
    @Override  
    public double getPreco() {  
        // Exemplo: produtos orgânicos têm 10% de desconto padrão  
        return super.getPreco() * 0.9;  
    }  
}  
  
// Polimorfismo em ação  
Produto p = new ProdutoOrganico("Tomate", 10.0);  
System.out.println(p.getPreco()); // Usa o getPreco() da subclasse
```

O comportamento real depende do tipo concreto do objeto em tempo de execução.

8. Interfaces

Interfaces definem **contratos** que classes devem cumprir.

```
public interface Calculavel {  
    double calcular();  
}
```

Elas são fundamentais para:

- Baixo acoplamento
- Inversão de dependência
- Arquiteturas flexíveis

Exemplo de redução de acoplamento com interfaces:

```
public interface PedidoRepository {
    void salvar(Pedido pedido);
}

public class PedidoRepositoryMySQL implements PedidoRepository {
    @Override
    public void salvar(Pedido pedido) {
        // Implementação concreta
    }
}

public class PedidoService {
    private final PedidoRepository repository;

    public PedidoService(PedidoRepository repository) {
        this.repository = repository; // Depende de uma abstração
    }

    public void finalizar(Pedido pedido) {
        // Regras de negócio
        repository.salvar(pedido);
    }
}
```

9. Composição vs Herança

- **Herança:** relação "é um"
- **Composição:** relação "tem um"

Boa prática:

Preferir composição à herança.

A composição gera sistemas mais flexíveis e evolutivos.

10. Acoplamento e Coesão

Acoplamento

Acoplamento representa o **nível de dependência entre classes ou módulos** de um sistema.

- **Alto acoplamento:** muitas dependências diretas, mudanças se propagam facilmente.
- **Baixo acoplamento:** poucas dependências, módulos mais independentes.

Exemplo (feira):

- Um `PedidoService` que depende diretamente de `MySQLConnection` → alto acoplamento.
- Um `PedidoService` que depende de uma interface `PedidoRepository` → baixo acoplamento.

Benefícios do baixo acoplamento:

- Facilita manutenção
 - Facilita testes
 - Permite evolução da arquitetura
-

Coesão

Coesão representa o **grau de relacionamento entre as responsabilidades de uma classe**.

- **Alta coesão:** classe tem responsabilidades bem definidas e relacionadas.
- **Baixa coesão:** classe faz muitas coisas diferentes e sem relação clara.

Exemplo (feira):

- Classe `Pedido` apenas com regras de pedido → alta coesão.
- Classe `SistemaFeira` com login, pedido, relatório e pagamento → baixa coesão.

Benefícios da alta coesão:

- Código mais legível
 - Menor complexidade
 - Melhor reutilização
-

Relação entre Acoplamento e Coesão

Boa arquitetura busca sempre:

Baixo acoplamento + Alta coesão

Esses dois conceitos são a **base estrutural de SOLID, GRASP e Arquitetura de Software**.

11. Erros Comuns em OO

- Classes Deus (muitas responsabilidades)
- Muitos getters/setters sem comportamento
- Herança excessiva
- Falta de encapsulamento

Esses erros impactam diretamente a arquitetura do sistema.

12. OO e SOLID

Os princípios SOLID (Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) dependem diretamente de uma boa modelagem OO:

- SRP — Single Responsibility Principle (Princípio da Responsabilidade Única) → classes com responsabilidades únicas
- OCP — Open-Closed Principle (Princípio Aberto/Fechado) → abstrações e extensibilidade
- LSP — Liskov Substitution Principle (Princípio da Substituição de Liskov) → herança correta

- ISP — Interface Segregation Principle (Princípio da Segregação de Interface) → interfaces coesas
- DIP — Dependency Inversion Principle (Princípio da Inversão de Dependência) → dependência de abstrações

Sem OO bem feita, não existe boa arquitetura.

Exemplos práticos (Feira Livre)

Os exemplos abaixo usam o mini projeto console para ilustrar cada princípio.

SRP — Single Responsibility Principle

Cada classe deve ter uma única responsabilidade.

- **Produto** foca em dados e validações de produto (nome/preço) — ver [feira-livre-java/src/feira/Produto.java](#).

```
public class Produto {  
    private String nome; // responsabilidade: dados de produto  
    private double preco; // validação de preço não-negativo  
    // get/set com validação: coeso e encapsulado  
}
```

- Anti - exemplo a evitar: colocar cálculo de **total()** do pedido dentro de **Produto** (mistura de responsabilidades).

OCP — Open-Closed Principle

Aberto para extensão, fechado para modificação.

- **ProdutoOrganico** estende **Produto** e sobrescreve **getPreco()** (sem modificar **Produto**) — ver [feira-livre-java/src/feira/ProdutoOrganico.java](#).

```
public class ProdutoOrganico extends Produto {  
    @Override  
    public double getPreco() { return super.getPreco() * 0.9; }  
}
```

- Extensão possível: criar `ProdutoPromocional` com outra política de preço, mantendo `Produto` intacto.

```
public class ProdutoPromocional extends Produto {  
    @Override  
    public double getPreco() { return super.getPreco() * 0.8; }  
}
```

LSP — Liskov Substitution Principle

Subtipos devem ser substituíveis pelos seus tipos base sem quebrar expectativas.

- Em `Main`, `Produto` pode referenciar `ProdutoOrganico` e o sistema continua correto — ver [feira-livre-java/src/feira/Main.java](#).

```
Produto p = new ProdutoOrganico("Tomate", 10.0); // usa como Produto  
double precoAplicado = p.getPreco(); // comportamento válido do subtipo
```

- Atenção: não violar invariantes do tipo base (ex.: não permitir preço negativo em subtipos).

ISP — Interface Segregation Principle

Prefira contratos pequenos/coerentes a interfaces “inchadas”.

- `PedidoRepository` tem um contrato mínimo `salvar(Pedido)` — ver [feira-livre-java/src/feira/PedidoRepository.java](#). Isso evita obrigar implementações a métodos desnecessários.

```
public interface PedidoRepository { void salvar(Pedido pedido); }
```

- Se surgirem necessidades distintas (ex.: consulta), pode - se criar `PedidoConsultaRepository` separado, mantendo coesão.

DIP — Dependency Inversion Principle

Depender de abstrações, não de concretos.

- **PedidoService** depende da interface **PedidoRepository**, injetada no construtor — ver [feira-livre-java/src/feira/PedidoService.java](#).

```
public class PedidoService {  
    private final PedidoRepository repository; // abstração  
    public PedidoService(PedidoRepository repository) { this.repository =  
repository; }  
    public double finalizar(Pedido pedido) { repository.salvar(pedido); return  
pedido.total(); }  
}
```

- Implementações podem variar (**PedidoRepositoryMemoria**, futuramente JPA) sem alterar o serviço — ver [feira-livre-java/src/feira/PedidoRepositoryMemoria.java](#).

13. OO e Arquitetura de Software

A OO é a base estrutural da arquitetura:

- Classes → componentes
- Objetos → serviços
- Interfaces → contratos arquiteturais

A qualidade da arquitetura depende da qualidade da modelagem OO.



Fluxo: Interface → Serviço → Repositório → Domínio

14. UML Essencial – Notação e Dicas

UML Essencial

Classes · Sequência · Casos de Uso · Notação · Mapeamento

A UML (Unified Modeling Language – Linguagem de Modelagem Unificada) é um conjunto de notações para representar sistemas de forma padronizada. Ela complementa OO, SOLID e arquitetura, ajudando a comunicar modelos e decisões.

Diagramas mais úteis na disciplina

- Diagrama de Classes: estrutura estática (classes, atributos, métodos, relacionamentos).
- Diagrama de Sequência: interação temporal entre objetos (mensagens e ordem).
- Casos de Uso: visão funcional (atores e objetivos do sistema).

Notações essenciais (Diagrama de Classes)

- Classe: retângulo com nome; membros por visibilidade (+ público, - privado, # protegido).
- Herança (generalização): seta com triângulo oco apontando para a superclasse.
- Interface: estereótipo «interface»; implementação com linha tracejada e triângulo.
- Associação: linha simples; multiplicidades 1, 0..1, * (muitos).
- Agregação/Composição: losango oco/cheio no lado do "todo".

Mapeamento UML → Código (Java)

- Classe → `class`; atributos → campos; operações → métodos.
- Herança → `extends`; Interface → `interface`; Implementação → `implements`.
- Visibilidade: + → `public`, - → `private`, # → `protected`.

Exemplo simples (Feira)

Relacionamentos principais:

- **Pedido** compõe **PedidoItem** (composição), associa-se a **Produto**.
- **ProdutoOrganico** herda de **Produto** (generalização).
- **PedidoService** depende de **PedidoRepository** (contrato/interface).

Exemplo visual (Mermaid – requer suporte de preview):



Syntax error in text
mermaid version 11.12.2

Diagrama de Sequência – exemplo (Fluxo de Finalização de Pedido)

Representa a interação temporal entre objetos da aplicação console:

```
Parse error on line 2:
...am      actor Usuario      participant Mai
-----^
Expecting 'SOLID_OPEN_ARROW', 'DOTTED_OPEN_ARROW', 'SOLID_ARROW',
'DOTTED_ARROW', 'SOLID_CROSS', 'DOTTED_CROSS', got 'NL'
```

Observação: em uma aplicação web (Spring), **Main** seria substituída por um **Controller**, mas a dinâmica entre **Service** e **Repository** permanece.

Quando usar UML

- Para comunicar modelo de domínio, regras e decisões arquiteturais entre equipes.
- Em checkpoints: incluir pelo menos um diagrama de classes e um de sequência.

Para uma visão de arquitetura de alto nível, consulte também o guia de C4: ver arquivo **C4-guidelines.md**.

15. Estudo de Caso Conceitual – Feira Livre

Exemplo de entidades:

- Feirante
- Produto
- Banca
- Pedido

Questões arquiteturais:

- Quem calcula o total do pedido?
- Onde ficam as regras de preço?
- Quem valida estoque?

Essas perguntas são resolvidas com boa OO.

16. Quiz de Revisão – Orientação a Objetos, Acoplamento e Coesão

Instruções

Responda às questões a seguir sem consultar o material. O objetivo é verificar a compreensão conceitual.

Questão 1

O que é Orientação a Objetos?

A) Um paradigma baseado apenas em funções. B) Um paradigma baseado em objetos que possuem estado e comportamento. C) Um framework de desenvolvimento. D) Uma linguagem de programação específica.

Questão 2

Qual a diferença entre classe e objeto?

A) Classe é um objeto abstrato. B) Objeto é um molde. C) Classe é um molde e objeto é uma instância. D) Não existe diferença.

Questão 3

O que é encapsulamento?

A) Herança de atributos. B) Ocultar detalhes internos e expor apenas o necessário. C) Criar muitas classes. D) Usar apenas métodos públicos.

Questão 4

Qual alternativa representa corretamente o conceito de herança?

A) Relação "tem um". B) Relação "usa um". C) Relação "é um". D) Relação "depende de".

Questão 5

O que é polimorfismo?

A) Capacidade de uma classe ter muitos atributos. B) Capacidade de diferentes objetos responderem à mesma mensagem. C) Capacidade de esconder dados. D) Capacidade de herdar múltiplas classes.

Questão 6

O que caracteriza **baixo acoplamento**?

A) Muitas dependências entre classes. B) Classes altamente dependentes. C) Poucas dependências e maior independência. D) Uso excessivo de herança.

Questão 7

O que caracteriza **alta coesão**?

A) Classe com muitas responsabilidades. B) Classe com responsabilidades bem definidas e relacionadas. C) Classe que depende de muitas outras. D) Classe com muitos métodos públicos.

Questão 8

Qual é a combinação desejável em sistemas bem projetados?

A) Alto acoplamento e alta coesão. B) Baixo acoplamento e baixa coesão. C) Alto acoplamento e baixa coesão. D) Baixo acoplamento e alta coesão.

Questão 9

Qual prática ajuda a reduzir acoplamento?

A) Depender de classes concretas. B) Usar variáveis globais. C) Programar contra interfaces. D) Concentrar lógica em uma única classe.

Questão 10

Por que acoplamento e coesão são importantes para arquitetura?

A) Porque melhoram a performance. B) Porque reduzem o número de linhas de código. C) Porque facilitam manutenção, testes e evolução do sistema. D) Porque eliminam a necessidade de documentação.

17. Gabarito do Quiz

1. B
2. C
3. B
4. C

- 5. B
 - 6. C
 - 7. B
 - 8. D
 - 9. C
 - 10. C
-

18. Conclusão

Orientação a Objetos não é sobre sintaxe, mas sobre:

- Modelar corretamente o domínio
- Separar responsabilidades
- Reduzir acoplamento
- Facilitar evolução

Ela é o **alicerce de toda a disciplina de Projeto e Arquitetura de Sistemas**.

19. Bibliografia Recomendada

- LARMAN, Craig. *Utilizando UML e Padrões*.
 - UML — Unified Modeling Language (Linguagem de Modelagem Unificada)
 - GAMMA et al. *Design Patterns*.
 - MARTIN, Robert C. *Arquitetura Limpa*.
 - FOWLER, Martin. *UML Essencial*.
-